Denormalization

counter caches, materialized views and keeping copies honest

Watch the 4-min video

In one line: denormalization stores data more than once, or stores precomputed answers, so hot reads get fast; you pay with extra writes, hot rows and the work of keeping every copy consistent.

1 Store the answer instead of recomputing it

Your social app shows a feed of posts, each with its like count. The likes table has **2 billion rows**. Running `COUNT(*)` for every post on every feed load is far too slow. The fix is to store the answer once and read it back. That is **denormalization**.

A **normalized** design stores each fact exactly once. A **denormalized** design deliberately duplicates some facts, or stores values derived from them. It trades extra write cost and consistency effort for read speed. It is a tool for a measured problem, not a default.

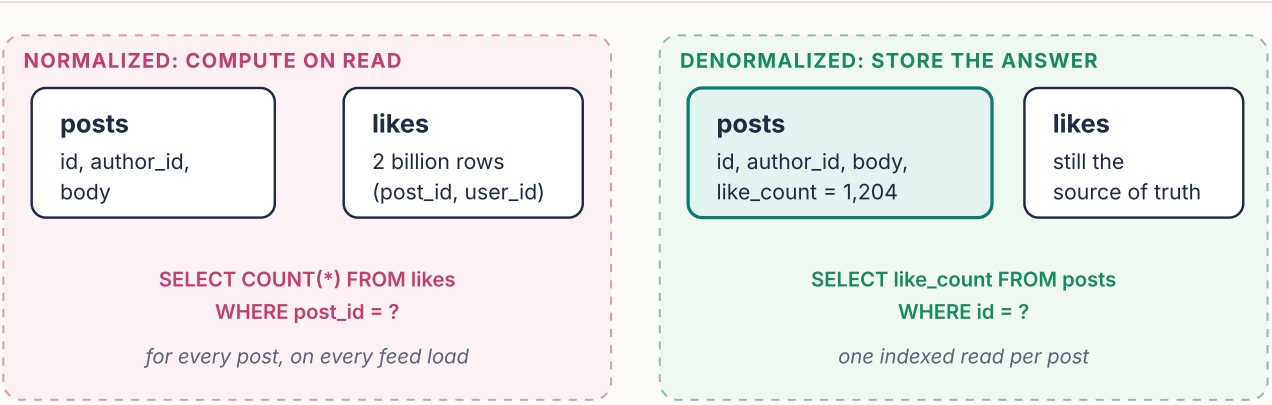


Figure 1. The likes table stays the source of truth. The counter on posts is a cached answer that the feed can read with one indexed lookup.

2 Four common forms

Form	Example	What it saves
Counter cache	`posts.like_count`, `users.follower_count`	a COUNT over a huge table
Embedded field	copy `author_name` into each post	a join to users on every feed read
Precomputed aggregate	daily revenue per product	a GROUP BY over all orders
Materialized view	the database runs a query and stores the result as a table	re-running an expensive report query

3 Example: a like counter done right

Keep a `like_count` column on posts. When someone likes a post, do both writes **in one transaction**: insert the like row and add one to the counter. Unlike deletes the row and subtracts one. Reading the feed is now one simple indexed read per post.

Make the increment **atomic in SQL**: `SET like_count = like_count + 1`. Never read the value into your application, add one and write it back. Two requests can read the same value and both write the same result, and one like is lost.

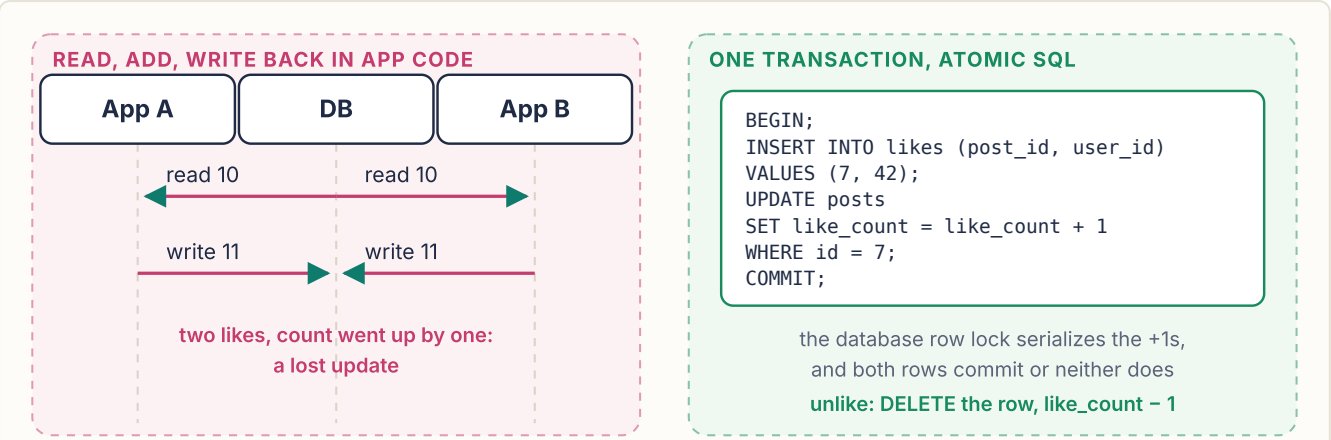


Figure 2. Left: read-modify-write in app code loses updates under concurrency. Right: the database applies each +1 under a row lock, inside the same transaction as the insert.

4 The costs are real

- **Write amplification.** Every like now writes two tables instead of one. An embedded author name means a rename touches every one of that user's posts.
- **Hot rows.** A viral post's counter may be updated thousands of times per second. Every update needs the same row lock, so writers queue up behind each other (lock contention).
- **Consistency.** The copies drift as soon as any code path forgets to update them: a bulk delete script, a new endpoint, a failed background job.

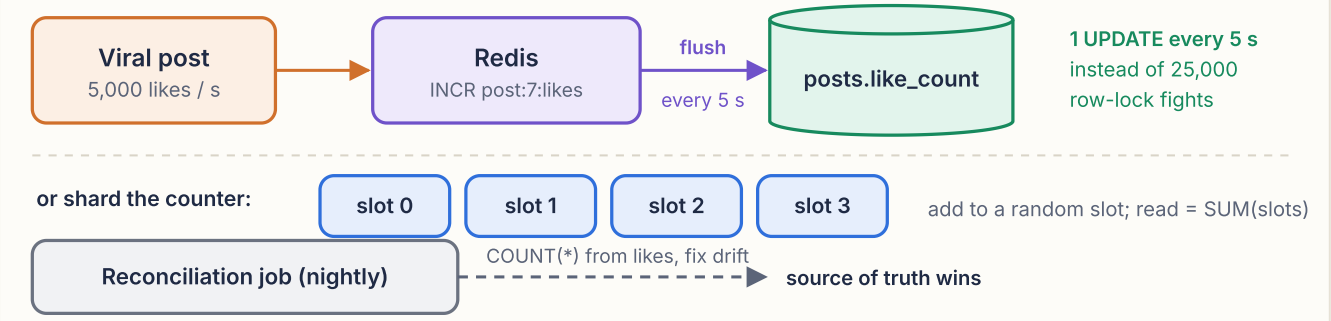


Figure 3. Two ways to tame a hot counter, plus the safety net. Buffering accepts that the shown count lags by a few seconds, which users never notice on a like button.

5 Keeping denormalized data consistent

- **Same database:** update source and copy in the **same transaction**, or use a **trigger** if you want the database itself to guarantee it, whatever code path does the write.
- **Copies in other services** (a search index, another team's database): **publish an event** and update them asynchronously. Accept **eventual consistency**: they lag by seconds.

- **Hot counters: buffer** increments in Redis and flush every few seconds, or **shard** the counter across several rows and sum them on read.
- **Always:** run a periodic **reconciliation job** that recomputes the truth from the source and fixes any drift.

WORKED EXAMPLE

A post goes viral at **5,000 likes/s**. Updating the counter row directly means 5,000 lock-holding UPDATEs per second on one row. With a Redis `INCR` and a flush every **5 s**, the database sees **1** UPDATE (`like_count = like_count + 25,000`) every 5 s. Alternatively, split the counter into **16** slot rows: each slot takes about 5,000 / 16 ≈ **310** updates/s, and a read sums 16 small rows. Either way, the likes table is still written once per like, so it stays the source of truth for the nightly reconciliation.

6 Materialized views

A **materialized view** is denormalization managed by the database. Postgres runs the query once and stores the result as a table. You refresh it on a schedule; `CONCURRENTLY` rebuilds it without blocking readers (Postgres needs a unique index on the view for that).

```
CREATE MATERIALIZED VIEW daily_revenue AS
  SELECT product_id, date_trunc('day', created_at) AS day, SUM(total) AS revenue
  FROM orders GROUP BY 1, 2;
CREATE UNIQUE INDEX ON daily_revenue (product_id, day);

-- every 5 minutes, from a scheduler:
REFRESH MATERIALIZED VIEW CONCURRENTLY daily_revenue;
```

They are great for dashboards and reports, where data a few minutes old is perfectly fine. They are a poor fit for a like button, which must change the moment you click.

COMMON MISTAKES

- Denormalizing before measuring. Add the copy only when a hot, read-heavy path is proven slow with joins or aggregates.
- Incrementing in application code (read, add, write back) instead of `like_count = like_count + 1`.
- Updating the copy outside the transaction, so a crash leaves the like without the count.
- No reconciliation job: drift is silent and only grows.

7 Key terms

Normalization: each fact stored once

Denormalization: deliberate duplication or precomputation for reads

Counter cache: stored count, e.g. like_count

Materialized view: stored result of a query, refreshed

Write amplification: one logical write causes several physical writes

Hot row: one row updated so often it causes lock contention

Reconciliation: recompute from the source and fix drift

Eventual consistency: copies agree after a short delay

8 Interview questions

Q1. When would you denormalize? BASIC

When a hot, read-heavy path is too slow with joins or aggregates, and you have measured it. Examples: feed like counters, author names embedded in posts, daily report aggregates. Only if you can afford the extra write per change.

Q2. How do you keep denormalized data consistent? BASIC

Same database: update both in one transaction or with a trigger. Other services: publish events and accept eventual consistency. Hot counters: buffer in Redis or shard. Reports: refresh materialized views. And always reconcile periodically against the source of truth.

Q3. Why use like_count = like_count + 1 instead of computing the new value in code? BASIC

Two requests that both read 10 would both write 11, losing a like. The atomic SQL increment is applied by the database under a row lock, so 2 likes always add 2.

Q4. A celebrity post gets 10,000 likes per second and the posts table locks up. What do you do? SCENARIO

Stop updating one row per like. Buffer increments in Redis and flush every few seconds (one UPDATE of +50,000 per 5 s), or shard the counter into, say, 20 rows at about 500 updates/s each and sum them on read. Keep writing the likes rows so the count can be reconciled.

Q5. You embed author_name in 50 million posts. A user renames themselves. What happens? SCENARIO

Every one of their posts must be rewritten, possibly thousands of rows. Do it asynchronously from a rename event in batches, accept that old posts show the old name for a while, or store only author_id and cache the user name instead.

Q6. The finance dashboard runs a 40-second GROUP BY over all orders. Fix it. SCENARIO

Create a materialized view of the aggregate, add a unique index, and REFRESH MATERIALIZED VIEW CONCURRENTLY every few minutes. The dashboard reads a small precomputed table in milliseconds; finance accepts data up to a few minutes old.

Q7. You found 0.1% of posts have the wrong like_count. Which value is right? TRICKY

The likes table, because it is the source of truth. Recompute COUNT(*) for those posts and overwrite the counters, then find the code path that skipped the update: a bulk script, a failed job, or a write outside the transaction.

Next: SD-E-004 · B-Tree Indexes Explained backendroadmap.com